LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

CHAIR OF THEORETICAL COMPUTER SCIENCE AND THEOREM PROVING



Designing a Competitive Proof Verification System for TPTP-Style Benchmarks

Alexander Giller

Master's Thesis
Computer Science

Supervisor: Prof. Dr. Jasmin Blanchette

Advisor: Tanguy Bozec

Submission Date: October 15, 2026

Disclaimer

I confirm that this thesis type is my own work and I have documented all sources and material used.

Munich, October 15, 2026 Author

Abstract

Nowadays automated theorem provers (ATPs) are highly effective, but the trustworthiness of their results depends on independent validation of the proof objects they produce. This thesis presents the design and implementation of a Python-based proof verifier for TPTP/TSTP derivations, developed according to the specifications for ProoVer 2026, which is the proof checking competition held as part of the FLoC Olympics 2026 in Lisbon (Cailler & Guilloud 2026).

Due to the fact, that false positives are by far the worst outcome in automated theorem proving, the competition's scoring scheme also punishes a wrongly accepted proof ten times harder than it rewards a correct acceptance (Cailler & Guilloud 2026). From the expected score a fail-closed policy is derived, where the verifier accepts a proof only if every step has been validated positively. Furthermore it only rejects the proof if it has found a concrete defect, and abstains otherwise. The verifier implements this policy as a modular pipeline of parsing and normalization, structural and dependency validation, and rule-specific inference checking. Structural, propositional, quantifier, and clausal rules and Skolemization (including freshness of the introduced symbols) are checked internally and produce diagnostics for each failing step. All remaining steps, that could not be verified internally are delegated to an external prover - in this case vampire - that only sees the premises of the step but not the whole proof.

The evaluation on 91 problem/proof pairs given by the ProoVer2026 competition committee verifies 91 of 91 proofs within the 30 seconds wall clock time limit for each proof, correctly rejects all 47 invalid proofs, accepts all 44 valid proofs and never wrongly accepts or rejects a proof. This thesis contributes a scoring-derived verdict policy with a soundness argument, a modular verification architecture, a practical implementation, and an empirical analysis of its performance and robustness under simulated competition conditions.

Keywords: Automated Theorem Proving, Proof Verification, TPTP, TSTP, Fail-Closed Checking, Skolemization, ProoVer 2026.

Contents

1 Introduction **1**
- 1.1 Motivation and Problem . . . 1
- 1.2 The ProoVer 2026 Competition . . . 1
- 1.3 Objective, Research Questions, and Scope . . . 2
- 1.4 Contributions and Structure . . . 2

2 Background **2**
- 2.1 Automated Theorem Proving and Proof Certification . . . 3
- 2.2 First-Order Logic . . . 3
- 2.3 Normal Forms, Skolemization, and Equisatisfiability . . . 4
- 2.4 The TPTP/TSTP Ecosystem . . . 4
- 2.5 Proof Objects, Inference Rules, and Soundness . . . 5
- 2.6 Competition Setting: ProoVer 2026 . . . 6
- 2.7 Resulting Goals . . . 7

3 Related Work **7**
- 3.1 Approaches to Proof Checking . . . 8
- 3.2 Systems of ProoVer 2026 . . . 8
- 3.3 GDV and Nörgler . . . 9
- 3.4 Proof Checking Beyond TPTP . . . 10
- 3.5 Positioning of This Thesis . . . 10

4 Requirements **11**
- 4.1 Task and Assumptions . . . 11
- 4.2 Scoring and Its Consequences . . . 11
- 4.3 Functional Requirements . . . 12
- 4.4 Non-Functional Requirements . . . 12
- 4.5 Traceability . . . 12

5 Design **12**
- 5.1 Design Principles . . . 13
- 5.2 Architecture Overview . . . 13
- 5.3 Data Model . . . 14
- 5.4 Global Checks . . . 14
- 5.5 Step Dispatch and the Checker Interface . . . 15
- 5.6 Rule Checkers . . . 16
- 5.7 Checking Skolemization . . . 16
- 5.8 Delegation to an External ATP . . . 17
- 5.9 Verdicts and Soundness Argument . . . 18
- 5.10 Resource Control . . . 19
- 5.11 Diagnostics . . . 19
- 5.12 Design Decisions and Alternatives . . . 20
- 5.13 Summary . . . 20

6 Implementation **20**
6.1 Overview . . . 20
6.2 Internal Representation . . . 21
6.3 Parser . . . 21
6.4 Driver and Dispatch . . . 22
6.5 Global Checks . . . 22
6.6 Structural, Propositional, and Quantifier Rules . . . 22
6.7 Skolemization . . . 23
6.8 Equality and Clausal Reasoning . . . 24
6.9 Delegation to the External ATP . . . 24
6.10 Resource Control . . . 25
6.11 Test Suite . . . 25
6.12 Tooling for Reporting and Evaluation . . . 25
6.13 Deviations from the Design . . . 25

7 Evaluation **26**
7.1 Setup . . . 26
7.2 Correctness and Score . . . 27
7.3 Rule Coverage (RQ1) . . . 27
7.4 Architecture Metrics (RQ2) . . . 27
7.5 Run Time and Memory (RQ3) . . . 28
7.6 Robustness (RQ3) . . . 28
7.7 Ablations . . . 29
7.8 Error Analysis . . . 29
7.9 Threats to Validity . . . 29
7.10 Summary . . . 30

8 Discussion **30**
8.1 Interpretation of the Results . . . 31
8.2 Scoring Policy and Its Limits . . . 31
8.3 Trusted Computing Base and Soundness . . . 31
8.4 Design Trade-offs . . . 32
8.5 Limitations . . . 33

9 Conclusion and Future Work **33**
9.1 Summary . . . 33
9.2 Answers to the Research Questions . . . 33
9.3 Contributions Revisited . . . 34
9.4 Future Work . . . 34

10 Notes on form **35**
10.1 Formatting . . . 35
10.2 Citation . . . 35
10.3 Bibliography . . . 36

11 General Addenda **37**
11.1 Detailed Addition 37

12 Figures **37**
12.1 Example 1 37
12.2 Example 2 37

Bibliography **40**

1 Introduction

1.1 Motivation and Problem

Automated theorem proving (ATP) is a key technology for formal reasoning in mathematics, software verification, and knowledge-based systems (**?**). As ATP systems grow more sophisticated, they also become more likely to contain bugs, so a bare answer such as "the conjecture is a theorem" is hard to trust. A common countermeasure is to let the prover emit a *proof object*, a derivation that justifies its answer step by step, and to validate it with a separate and much simpler program, the *proof checker*. Trust is thereby moved from a large, fast-changing prover to a small and well-understood checker.

In the TPTP ecosystem (Sutcliffe 2024), derivations are exchanged in the TSTP format (Sutcliffe 2010, 2007). A TSTP derivation is a directed acyclic graph of formulas, where each node records the inference rule that produced it and the premises it depends on. Provers differ, in the rules they use, in how precisely they annotate steps, and in how they treat transformations such as clausification and Skolemization. A checker must therefore be sound, in that it never accepts an invalid step, while also covering the rules that occur in real prover output and staying within strict time and memory limits. The challenge can be divided into two separate parts. For one logical correctness at the step level has to be ensured and on the other hand engineering robustness at the workload level has to be maintained.

The established checker for TSTP derivations is GDV (Sutcliffe 2006b,c,a,d), where the inference verifications use external trusted ATP systems, which verify each individual inference step based on the SZS status (Sutcliffe 2008). This makes it general, but also slow and dependent on external systems. Therefore newer checkers address this issue by treating frequent inference rules and transformations internally and by rejecting wrong steps with model finders (Chapter 3). What is still missing is a systematic account of how a checker should behave when it cannot decide a step, because the cost of a wrong answer is not symmetric. This thesis derives such a policy from the scoring scheme of a proof checking competition and builds a checker around it.

1.2 The ProoVer 2026 Competition

This work is carried out in the context of ProoVer 2026, the first edition of this proof checking competition held as part of the FLoC Olympics 2026 in Lisbon (Cailler & Guilloud 2026). Like the CASC competition for provers (Sutcliffe 2016), ProoVer focuses on the TPTP ecosystem, but the systems under evaluation are checkers: they receive a FOF problem and a TSTP proof and must classify the proof as valid or invalid, or give up. Some of the proofs contain deliberate, subtle errors. Correct answers are rewarded, and wrongly accepting an invalid proof is penalized most heavily(Cailler & Guilloud 2026). The task, the scoring scheme, and the resource limits are described in detail in Chapter 4, and the structure of the benchmark files in Section 2.6. In this thesis, *competitive* means that the verifier achieves a high total score under the ProoVer 2026 scoring scheme (Section 4.2) within the per-proof time limit, with the average run time as a secondary criterion.

1.3 Objective, Research Questions, and Scope

The objective of this thesis is to design and implement a Python-based verifier for TSTP derivations that is sound and competitive in this sense. Soundness means that each accepted inference step is justified by the formal semantics of its rule. The verifier ensures this by syntactic rule checks, dedicated checks for multiple steps, with the focus lying on Skolemization. For the remaining steps, an external prover, namely Vampire 5.0.1, is used, and it accepts a proof only if every step has been validated positively. It is organized as a modular pipeline of parsing and normalization, dependency and structural validation, and rule-specific inference checking. Furthermore, it only targets FOF problems and the inference rules occurring in the ProoVer 2026 benchmarks. Steps with rules outside the supported set are delegated to the external prover, and if that is impossible they lead to abstention rather than acceptance. This research is structured around three central research questions.

RQ1 Which inference rules and formula fragments must a verifier support to cover the ProoVer 2026 benchmarks? *Measure:* share of steps that are validated internally, by the external prover, or not at all.

RQ2 How can a modular architecture keep the trusted part of the verifier small while remaining extensible? *Measure:* size and test coverage of the individual checkers, and the amount of work required to add a rule.

RQ3 How do correctness, running time, and memory consumption behave on realistic and on malformed or inconsistent derivations? *Measure:* score, verdict matrix, and run time on the benchmark set and on mutated proofs.

1.4 Contributions and Structure

The contributions of this thesis are: (i) a verdict policy for proof checking that is derived from the competition's scoring scheme and results in a three-valued, fail-closed verdict (Chapters 4 and 5); (ii) a modular verification pipeline for TSTP derivations with rule-local checkers, a dedicated Skolemization check, and a restricted ATP fallback, together with its Python implementation and diagnostics that identify failed steps (Chapters 5 and 6); and (iii) an evaluation on the ProoVer 2026 benchmarks and a suite of mutated proofs showing [TODO: key result, e.g. X of Y proofs verified within the limits, no wrong acceptance, and the achieved score], together with an analysis of the remaining limitations (Chapters 7 and 8).

The remainder of the thesis is organized as follows. Chapter 2 introduces the background on first-order logic and the TPTP/TSTP formats, and Chapter 3 discusses related work. Chapter 4 derives the requirements, Chapters 5 and 6 present the architecture and its implementation, and Chapter 7 reports the experimental results. Chapter 8 discusses implications and limitations, and Chapter 9 answers the research questions and concludes with future work.

2 Background

This chapter provides the technical foundations for the rest of the thesis. Section 2.1 motivates proof certification. Sections 2.2 and 2.3 recall first-order logic and the normal-form transformations

that appear in derivations. Sections 2.4 and 2.5 introduce the TPTP/TSTP ecosystem and define what it means for a checker to be sound. Section 2.6 describes the structure of the ProoVer 2026 benchmark files, and Section 2.7 summarizes the resulting goals.

2.1 Automated Theorem Proving and Proof Certification

Automated theorem provers search for formal derivations that establish the validity or unsatisfiability of logical statements. Their search procedures combine many techniques, for example saturation, clause selection heuristics, and indexing, and are correspondingly complex. Trusting such a system as a whole is difficult, because a single implementation error can lead to a wrong answer without any visible symptom (**?**).

Proof verification addresses this issue by separating *proof search* from *proof checking*. The prover generates a proof object, and an independent checker validates each inference step against a trusted set of rules. The set of components that must be correct for an accepted result to be trustworthy is called the *trusted computing base* (TCB). For a proof checker, the relevant property is that every accepted proof is valid. The TCB then consists of the checker itself, together with any external tool it relies on, whereas the prover that produced the proof is not part of it. We do not count the underlying language runtime, operating system, and hardware, as they are assumed to be correct and are not specific to the checker. Therefore, proof verification shrinks the TCB from the prover to the checker, which is typically orders of magnitude smaller (**??**).

Additionally, separating search from checking has practical benefits beyond trust. It improves diagnosability, because instead of a binary success or failure result, users obtain explicit intermediate steps that can be replayed and audited. Furthermore, it also makes benchmark comparisons less dependent on solver-specific implementation details, which is crucial for benchmark-driven research.

2.2 First-Order Logic

The derivations considered in this thesis are over classical first-order logic. We recall only the notions needed within the scope of this thesis, for a full treatment see Fitting (1996).

Syntax. A signature consists of function symbols and predicate symbols with fixed arities; constants are function symbols of arity zero. *Terms* are built from variables and function symbols. An *atom* is a predicate symbol applied to terms (or an equation $s = t$). *Formulas* are built from atoms, $\top$ and $\bot$ (written `$true` and `$false` in TPTP), the connectives $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$, and the quantifiers $\forall$ and $\exists$. A formula without free variables is a *sentence*. A *literal* is an atom or its negation, and a *clause* is a disjunction of literals.

Semantics. An interpretation assigns a domain, a function to each function symbol, and a relation to each predicate symbol. A sentence is *satisfiable* if some interpretation makes it true, and a set of sentences Γ *entails* a sentence φ, written $\Gamma \models \varphi$, if every interpretation that satisfies all of Γ also satisfies φ. Entailment and unsatisfiability are connected by

$$\Gamma \models \varphi \quad \text{iff} \quad \Gamma \cup \{\neg\varphi\} \text{ is unsatisfiable.} \tag{1}$$

Most ATP systems exploit this *refutation principle*: to prove a conjecture, they negate it, add it to the axioms, and derive a contradiction. This explains why TSTP proofs of theorems typically end in $\bot$ and contain a negated conjecture.

2.3 Normal Forms, Skolemization, and Equisatisfiability

Many provers do not work on arbitrary formulas, but first transform them into a normal form, usually conjunctive normal form (CNF) of a quantifier-free matrix. This *clausification* consists of eliminating $\Rightarrow$ and $\Leftrightarrow$, pushing negations inward (negation normal form), renaming variables apart, moving quantifiers outward (prenex form), eliminating existential quantifiers, and distributing $\vee$ over $\wedge$.

The elimination of existential quantifiers is called *Skolemization*. An existential variable y in the scope of universal variables $x_1, \ldots, x_n$ is replaced by a term $f(x_1, \ldots, x_n)$ with a fresh *Skolem function* f (a fresh constant if $n = 0$):

$$\forall x_1 \ldots \forall x_n \, \exists y \, \varphi(x_1, \ldots, x_n, y) \;\rightsquigarrow\; \forall x_1 \ldots \forall x_n \, \varphi(x_1, \ldots, x_n, f(x_1, \ldots, x_n)).$$

The result is in general *not* logically equivalent to the original formula, since it mentions the new symbol f. It is only *equisatisfiable*: the original formula is satisfiable if and only if the Skolemized formula is. For refutation-based proofs this suffices, because only unsatisfiability matters (Equation 1). It has, however, a consequence for checking: a Skolemization step cannot be validated by an entailment check between parent and conclusion. The checker must instead verify the shape of the transformation and the *freshness* of the introduced symbol, i.e. that f does not occur elsewhere in the derivation (**?**). Symmetrically, instantiating a universal formula with a fresh constant, as in Herbrand-style reasoning, introduces new symbols that must be tracked. This is why the symbol-introduction annotations in ProoVer 2026 files (Section 2.6) are relevant for verification.

2.4 The TPTP/TSTP Ecosystem

The TPTP (Thousands of Problems for Theorem Provers) library provides a standardized collection of benchmark problems across multiple logics and domains. It has become a central infrastructure for evaluating ATP systems under comparable conditions (**?**). Problems are encoded in a common syntax and distributed with metadata that supports reproducible experiments. Besides first-order formulas (`fof`), the language includes clause normal form (`cnf`) and typed and higher-order variants. This thesis concentrates on `fof`, the language of the ProoVer 2026 benchmarks (Chapter 4).

At a syntactic level, a first-order TPTP input file is typically a sequence of declarations of the form:

```
fof(Name, Role, Formula, SourceOrAnnotation).
```

The `Name` is a unique identifier within the file, `Role` describes the semantic status of the statement (for example `axiom` or `conjecture`), and `Formula` contains the logical content. The final field records provenance or additional annotations. In benchmark files, source references often point back to a problem file and formula identifier, which makes provenance explicit and machine-checkable.

TSTP (Thousands of Solutions from Theorem Provers) complements TPTP by specifying a structured format for solver outputs, including proof traces. A TSTP derivation typically consists

```
fof(a1,axiom, ?[X]: ![Y]: ?[Z]: ![W]: r(X,Y,Z,W),
    file('Problems/PRV000+1.p',a1) ).
fof(c,conjecture, ?[X]: ![Y]: ?[Z]: ![W]: r(X,Y,Z,W),
    file('Problems/PRV000+1.p',c) ).
fof(neg,negated_conjecture,
    ![A]: ?[B]: ![C]: ?[D]: ~r(A,B,C,D),
    inference(negated_conjecture,[status(cth)],[c]) ).
fof(bot,plain, $false,
    inference(consequence,[status(thm)],[neg,a1]) ).
```

Figure 1: The ProoVer 2026 proof `PRV000+1.s` (comments and SZS delimiters omitted). Axiom and conjecture are imported from the problem file, `neg` negates the conjecture, and one `consequence` step derives `$false` from `neg` and `a1`.

of named formulas or clauses, inference annotations, and references to parent steps. This explicit dependency structure enables external verification, debugging, and post-hoc analysis (**?**).

In proof output, the fourth field is usually an `inference(...)` annotation rather than a static source reference:

```
inference(RuleName, [StatusAndMeta], [Parent1, Parent2, ...])
```

Here, `RuleName` identifies the transformation, the middle list stores status information and optional metadata (for example newly introduced symbols), and the parent list defines the local dependency graph. This structure makes TSTP proof objects suitable for independent replay by a checker that does not rely on internal prover state.

SZS ontology. Solver results are communicated through the SZS ontology (**?**), which standardizes status values such as `Theorem` or `Unsatisfiable` and the delimiters of proof output (`% SZS output start Proof` and `% SZS output end Proof`). A checker uses these delimiters to locate the derivation within a larger solver output.

2.5 Proof Objects, Inference Rules, and Soundness

A proof object in the TSTP style represents a directed acyclic derivation where each node is justified by an inference rule and references to prior nodes (or axioms). Figure 1 shows the smallest proof of the ProoVer 2026 benchmark set, which already needs a semantic check of its last step.

Verifying such a proof requires:

- parsing and normalizing formula representations,
- resolving symbol and variable scopes correctly,
- validating rule applications (e.g., propositional, clausal, instantiation, and structural steps),
- ensuring dependency consistency and acyclicity where required.

From an implementation viewpoint, these tasks are tightly coupled. For example, a checker can only validate an `instantiate` step after resolving quantifier structure in the parent formula; similarly, validating `skolemize` requires both formula transformation checks and side-condition checks on generated symbols. As a result, practical proof verification combines syntactic processing, semantic rule checks, and graph-level consistency checks.

Step status. The status field distinguishes the logical strength of a step. A step with status `thm` claims that its conclusion is entailed by its parents, whereas a step with status `esa` (equisatisfiable) only claims preservation of satisfiability, as for Skolemization (**?**). In the ProoVer 2026 files the statuses `thm`, `esa`, and `cth` occur; `cth` marks the step that negates the conjecture. A checker must apply the appropriate criterion to each step: entailment for `thm` steps, and shape and side conditions for equisatisfiability-preserving steps.

Soundness. Let Γ be the set of axioms and negated conjecture of a derivation, and let D be a derivation of $\bot$ from Γ. A checker is *sound* if it accepts D only if Γ is unsatisfiable, and hence, by Equation 1, the conjecture is entailed by the axioms. This property follows from *step-wise soundness*, which has to be stated separately for the two kinds of steps. A step with status `thm` is sound if its conclusion is entailed by its premises. A step with status `esa` is sound if it preserves satisfiability: every model of its premises can be extended, by interpreting the fresh symbols it introduces, to a model of the premises and the conclusion. If all steps are sound and the dependency graph is acyclic, an induction over the graph yields the invariant that Γ together with all formulas derived so far is satisfiable whenever Γ is. This is impossible once $\bot$ has been derived, so Γ is unsatisfiable. Section 5.9 carries out this argument for the verifier. In practice, systems adopt conservative behavior for unsupported constructs: unknown inference rules are rejected, flagged, or isolated to avoid unsound acceptance. This design choice trades permissiveness for trustworthiness, which is generally desirable in certification-oriented workflows (**?**). A complementary property is *completeness* on a given benchmark set, i.e. that all valid derivations are accepted. It is not a correctness requirement, but it determines how many benchmarks a checker can verify; the thesis therefore reports it separately as coverage.

2.6 Competition Setting: ProoVer 2026

Competition scenarios such as ProoVer 2026 impose additional engineering constraints beyond pure logical correctness. A competitive verifier must process diverse benchmark sets, cope with differences in prover output style, and provide stable performance under time and memory limits. Architecture decisions such as modular rule checkers, efficient internal data structures, and clear diagnostics are therefore not merely implementation details; they directly affect scientific usability and benchmark competitiveness.

The ProoVer 2026 benchmark files follow a regular TSTP-style profile that is particularly useful for discussing verification requirements. Each proof file starts with comment metadata and explicit SZS delimiters:

```
% SZS output start Proof
...
% SZS output end Proof
```

Inside these delimiters, entries are given as `fof(...)` statements. Typical files begin with `axiom` and `conjecture` items linked to the corresponding problem source, followed by derived `plain` steps, a `negated_conjecture` step, and finally a contradiction step deriving `$false` via `inference(consequence,...)`.

Two concrete examples illustrate the spectrum. The small file `PRV000+1.s` has a compact structure: source formulas, negated conjecture, and one final consequence step to contradiction. In contrast, `PRV010+1.s` contains a long derivation with many rule families, including `excluded_middle`, `double_negation`, `instantiate`, `add_hypothesis`, `existential_gen`, `remove_double_negation`, `split_conjunct`, `reflexivity`, `rename_variable`, `weaken`, `commute`, `skolemize`, and `consequence`. This diversity motivates a modular checker architecture with explicit per-rule validation logic. Table 7 in Chapter 5 lists these rules with their intended reading, and Table 6 groups them by the principle of their check.

Another ProoVer 2026-specific aspect is metadata for generated symbols. Skolemization steps can carry annotations such as:

```
new_symbols(skolem,[sK0]), skolemize(X1,sK0)
```

and some instantiation steps include Herbrand-symbol annotations like:

```
new_symbols(herbrand,[m0])
```

These annotations are not merely decorative; they describe symbol-introduction events that a robust checker should parse and validate consistently against the transformed formula, in particular with respect to the freshness condition from Section 2.3. Section 5.7 describes how the verifier checks `skolemize` steps and their `new_symbols` annotations.

2.7 Resulting Goals

Designing a practical proof verifier therefore requires balancing three goals:

1. high assurance through explicit rule-level validation,
2. broad compatibility with heterogeneous TPTP/TSTP-style derivations,
3. efficient execution on large benchmark collections.

These goals define the technical context in which the present thesis is positioned and motivate the requirements and design decisions in the following chapters.

3 Related Work

This chapter positions the thesis with respect to existing work on checking proofs of automated reasoners. Section 3.1 organizes the approaches by how a checker establishes the validity of an inference step. Section 3.2 surveys the systems of ProoVer 2026, which address the same task as this thesis. Section 3.3 discusses the two most closely related systems, GDV and Nörgler, in more detail. Section 3.4 briefly covers proof checking outside the TPTP ecosystem, and Section 3.5 states how this thesis differs from the work discussed.

3.1 Approaches to Proof Checking

Checkers for TSTP derivations can be grouped by how they decide that a single inference step is correct.

Semantic checking with external provers. The checker verifies the global and local structure of the derivation and then asks a trusted ATP to confirm that each step's conclusion follows from its premises. Incorrect steps may additionally be refuted by a model finder, which searches for a countermodel. This approach reuses mature provers and is applicable to any inference rule, but it increases the trusted computing base by the ATP and depends on its running time.

Internal replay. The checker contains its own implementation of the rules, for example of resolution or superposition, and replays each step without external tools. This reduces external dependencies and can be fast, but every supported rule has to be implemented and trusted.

Translation to a logical framework or proof assistant. The derivation, or the output of an ATP checking its steps, is translated into a proof term or script that a small kernel of a proof assistant or logical framework verifies. This yields strong guarantees, since only the kernel must be trusted, at the cost of translation effort and run time.

Hybrids. Practical systems often combine these strategies, for example by checking Skolemization steps structurally and delegating entailment steps to an ATP. The systems in Section 3.2 illustrate the full spectrum.

3.2 Systems of ProoVer 2026

ProoVer 2026 is the first edition of the ProoVer competition and was held on 27 July 2026 in Lisbon, alongside CASC-J13 at FLoC 2026 (Cailler & Guilloud 2026). Systems receive FOF problems with TSTP proofs and must classify each proof as verified, failed, or uncertain; some proofs are deliberately flawed. Correct verdicts earn points, abstention scores zero, and wrongly accepting a bad proof is penalized most heavily (PLA 2026). Ten systems were entered, eight in the regular division and two (GDV and GDV-LP) in a demonstration division. Table 1 summarizes them according to their system descriptions in the proceedings (PLA 2026). [TODO: add the final ranking and scores once the official results are available, e.g. from the CASC-J13 result pages.]

Three observations are relevant for this thesis. First, most entrants delegate entailment steps to external provers (E, Vampire) and use model finders such as Mace4 to reject wrong steps; only CheckProof and GAPT rely on internal reasoning engines. Second, Skolemization is a recurring special case: several systems, including ProofGuard and PyCheck, check it structurally, with a freshness check on the introduced symbol, instead of via entailment, in line with the discussion in Section 2.3. Third, two systems, ProofGuard and PyCheck, are written in Python. They are therefore the closest points of comparison for the implementation language and are discussed in Section 3.5.

Table 1: Systems entered in ProoVer 2026, as described in their system descriptions. "Ext." lists external tools the system relies on at run time.

System	Language	Core approach	Ext. tools
CheckProof 0.1	C	Internal replay; own saturation engine	none stated
GAPT 2.20	Scala	Import as LK proof; internal prover Escargot	none (internal)
GDV 2.0	C	Four-phase semantic verification	remote ATPs via SystemOnTPTP
GDV-LP 2.0	C, OCaml tools	GDV extended by LambdaPi translation	ZenonModulo, lambdapi, other Dedukti checkers
mrs-proover 0.2.0	Rust	Structural checks plus ATP ladder	mrs (internal), E, Vampire
Nörgler 1.1	Scala	Semantic verification with countermodel search	E, Vampire, Mace4
ProofCheck 1.0	C++	Structural checks plus internal clausal engine	E, Mace4, Vampire
ProofGuard 1.0	Python 3	Conservative checker with internal structural checks	E (bundled)
PyCheck 0.1	Python (reuses PyRes)	Structural and semantic checks	E
VaLeaDate 0.1	Scala, Lean	Per-step Vampire call; whole proof checked in Lean	Vampire, Lean (VampLean)

3.3 GDV and Nörgler

GDV. The Derivation Verifier GDV is the established tool for checking TSTP derivations (Sutcliffe 2006e). It verifies a derivation in four phases: structural checks, checks of the leaf formulas against the problem, rule-specific checks, and finally the verification of inference steps by calling trusted ATPs via SystemOnTPTP. GDV is thus the reference for the semantic checking approach. Its dependence on external provers makes it comparatively slow and ties its trust to the ATPs it calls. GDV-LP (PLA) addresses the latter by translating proofs of individual steps into LambdaPi terms, so that the ATP no longer needs to be trusted.

Nörgler. Nörgler (Taprogge 2026) is a light-weight checker for TSTP refutations that follows the semantic approach of GDV. It checks global and local structure, logical correctness, and faithfulness to the problem, uses an internal procedure for Skolemization steps, external provers for entailment, and a model finder to reject incorrect steps. In the evaluation reported by its authors, it verified the same 1972 correct proofs as GDV, rejected more of 1952 falsified proofs than GDV in its best configurations, and was faster on average (0.084 s versus 0.542 s per proof in its fastest mode). These results show that a careful treatment of Skolemization and parallel checking can close much

Table 2: Comparison of the closest related systems with this thesis. [TODO: fill in the last column according to the implementation; verify the other entries against the system descriptions.]

Dimension	GDV	Nörgler	ProofGuard / PyCheck	This thesis
Implementation language	C	Scala	Python	Python
Needs external ATP for entailment	yes	yes	yes (E)	yes (Vampire)
Internal Skolem check	–	yes	yes	yes
Modular per-rule components	[TODO]	[TODO]	[TODO]	yes
Diagnostics per failed step	[TODO]	[TODO]	[TODO]	yes

of the performance gap to internal replay, and they provide a useful baseline for the evaluation in Chapter 7. [TODO: compare on the same benchmark set if possible.]

3.4 Proof Checking Beyond TPTP

The idea of independently checking solver output is much older than ProoVer and is established in neighboring areas. For propositional satisfiability, proof formats such as DRAT and LRAT allow solvers to produce certificates that small verified checkers validate (Wetzler et al. 2014). For SMT, the Alethe format and checkers such as Carcara play this role (Schurr et al. 2021). For first-order proofs, the Dedukti logical framework and its translators, for example from Zenon Modulo or Vampire output, enable verification by a small kernel (Delahaye et al. 2013). In interactive theorem proving, proof reconstruction replays the output of external provers inside the kernel of a proof assistant, as in Sledgehammer for Isabelle/HOL (Böhme & Nipkow 2010). GAPT (Ebner et al. 2016) provides infrastructure for importing and transforming proofs of various provers. These approaches generally give stronger guarantees than the direct checking considered here, but they require a proof format that is far more detailed than TSTP, or additional translation steps. In contrast, TSTP derivations contain coarse inference steps whose justification is left to the checker, which is the setting of this thesis.

3.5 Positioning of This Thesis

Table 2 compares this thesis with the closest systems along the dimensions that matter for its goals.

The discussion above shows that checking TSTP derivations is not a new problem and that several competing designs exist. This thesis does not claim to be better than these systems in general. Instead, it explores a particular trade-off: a small, modular Python checker in which every rule check is an independently testable function, internal checks are tried before any external prover is called, and the verdict policy is derived explicitly from the competition's scoring scheme (Section 4.2). Compared with GDV and Nörgler, which send most entailment steps to external provers, it validates the structural, propositional, quantifier, and Skolemization steps itself and uses an ATP only as a fallback. This reduces run time and the dependence on the ATP, but increases the amount of checker code that has to be trusted (Chapter 8). Compared with the other Python-based entrants ProofGuard and PyCheck, the differences lie in [TODO: verify against their system descriptions, e.g. rule coverage, diagnostics, test strategy]. The evaluation in Chapter 7 substantiates these differences with measurements.

4 Requirements

This chapter derives the requirements for the verifier from the ProoVer 2026 task and from the properties of TSTP derivations discussed in Chapter 2. Section 4.1 fixes the task and its assumptions. Section 4.2 analyses the scoring scheme, which determines how the verifier should behave when it is unsure. Sections 4.3 and 4.4 list the functional and non-functional requirements. Section 4.5 relates them to the components and tests of the implementation.

4.1 Task and Assumptions

The verifier receives a FOF problem file and a TSTP proof file and must decide whether the proof is a valid refutation of the negated conjecture, i.e. a derivation of `$false` from the axioms and the negated conjecture (Cailler & Guilloud 2026). The proofs use the roles `axiom`, `conjecture`, `negated_conjecture`, and `plain` and the statuses `thm`, `esa`, and `cth`. A part of the benchmark proofs contains deliberate, subtle errors. Only a small number of rules is specified by the organizers: Skolemization, the negated conjecture, and the import of axioms and conjectures. All other steps may be checked by an external ATP, but only if they have status `thm` or `cth` and only with the step's premises. The organizers state that handling edge cases is part of the task.

Let P be a problem with axioms A and conjecture C, and let a proof π be a finite list of annotated formulas. The verifier must decide whether π is a *valid refutation* of P: (i) every `axiom` and `conjecture` step of π is faithful to P; (ii) the dependency graph of π is acyclic and C is used only by the step that negates it; (iii) every other step is sound in the sense of Section 2.5; and (iv) π derives `$false`. It answers *verified*, *failed*, or *uncertain*. The competition input is restricted to FOF; CNF and typed or higher-order input are out of scope. We make the following assumptions:

- The input files are syntactically close to the TPTP BNF, but may be large (thousands of steps) and deeply nested.
- The problem file is trusted, the proof file is not.
- An external ATP and a model finder may or may not be available at run time; the verifier must behave safely in both cases.

4.2 Scoring and Its Consequences

In ProoVer 2026 a correct rejection of an invalid proof scores $+2$, a correct acceptance of a valid proof $+1$, giving up 0, a wrong rejection -1, and a wrong acceptance -10 (PLA 2026). Ties are broken by the average time on solved problems. The asymmetry has a direct consequence for the verifier's behavior in uncertain situations. Let p be the probability that a given proof is valid, given everything the verifier has found out. The expected scores of the three possible answers are

$$\begin{aligned} E[\text{accept}] &= p \cdot 1 + (1-p) \cdot (-10) = 11p - 10, \\ E[\text{reject}] &= p \cdot (-1) + (1-p) \cdot 2 = 2 - 3p, \\ E[\text{abstain}] &= 0. \end{aligned}$$

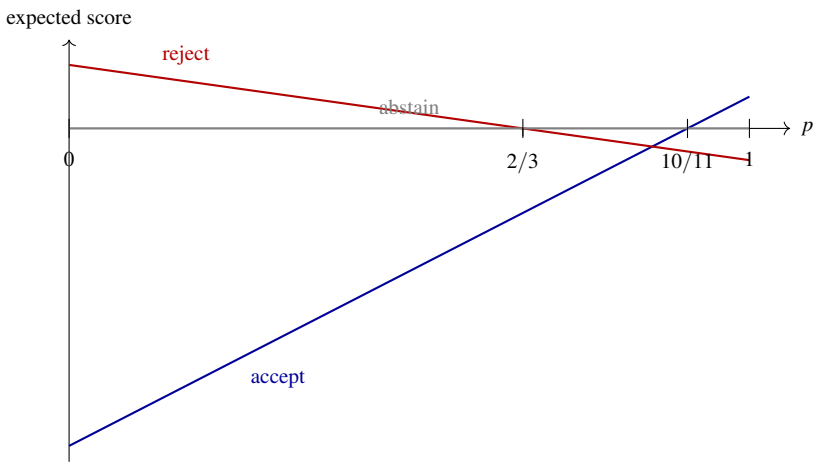


Figure 2: Expected score of accepting, rejecting, and abstaining as a function of the probability p that the proof is valid, under the ProoVer 2026 scoring scheme. Accepting beats abstaining only for $p > 10/11$, rejecting only for $p < 2/3$.

Accepting is thus better than abstaining only if $p > 10/11 \approx 0.91$, and rejecting is better than abstaining only if $p < 2/3$. For $2/3 < p < 10/11$, giving up is optimal. The verifier therefore must not guess: it accepts only if every step has been positively verified, it rejects only if it has found a concrete defect, and it abstains in all other cases, for example if a required tool is missing or a time limit is hit. A failed rule-specific check counts as a concrete defect only for rules without an ATP fallback; otherwise the step stays unresolved until the fallback has decided (Section 5.8). Figure 2 visualizes the three expected scores. The probability p is of course not observable; the analysis does not compute it but motivates the asymmetric thresholds and the fail-closed policy that gives rise to the soundness requirement below.

4.3 Functional Requirements

Table 3 lists the functional requirements. Each requirement has an identifier that is used for traceability in Section 4.5 and in the evaluation.

4.4 Non-Functional Requirements

4.5 Traceability

Table 5 shows which module implements each requirement and which test files exercise it. It is the basis for the claim that the requirements are covered and is revisited in Chapter 7.

5 Design

This chapter presents the architecture of the verifier. It describes components, data model, and decision rules independently of implementation details, which follow in Chapter 6.

Table 3: Functional requirements.

ID	Requirement
FR1	Parse FOF problem and proof files, including roles, `file(...)` sources, `inference(...)` annotations with status, parents, `new_symbols`, and `skolemize` information.
FR2	Check that each `axiom` and `conjecture` in the proof is alpha-equivalent to the formula of the cited problem file and role.
FR3	Check that the `negated_conjecture` step has status `cth`, has a `conjecture` as parent, and its formula is the negation of that conjecture.
FR4	Check that all cited parents exist and that the dependency graph is acyclic.
FR5	Check that the `conjecture` is not used as a premise by any step other than the negation step.
FR6	Check Skolemization steps: status `esa`, exactly one new Skolem symbol, a Skolem term that depends exactly on the universal variables in scope, a conclusion that results from replacing the eliminated variable by this term, and freshness of the symbol with respect to the problem and earlier steps.
FR7	Check each remaining inference step with a rule-specific checker whenever one exists (structural, propositional, quantifier, junction, clausal, and equational rules).
FR8	For steps without a rule-specific checker, or where a syntactic check fails on a known rule, delegate the step to an external ATP, but only if its status is `thm` or `cth` and only with the step's premises.
FR9	Check that the derivation ends in `$false`.
FR10	Produce one of three verdicts per proof: verified, failed, or uncertain.
FR11	Report, for each failing or uncertain step, its name, the rule, and a human-readable reason.

5.1 Design Principles

Four principles guide the design, each derived from the requirements of Chapter 4.

Fail closed (NFR1). Acceptance is the only outcome that requires positive evidence. Every error, absence of a tool, or unknown construct leads to rejection or abstention, never to acceptance.

Rule locality (NFR6, NFR7). Each inference rule is checked by a separate function that sees only the step and its parents and returns a list of issues. Rule checkers do not know of each other.

Cheap checks first (NFR2). Syntactic checks run before any external tool is called. An ATP is only a fallback for steps that cannot be validated syntactically.

Diagnostics as data (NFR5). Checkers return structured issues instead of printing or raising, so that reports, summaries, and tests use the same information.

5.2 Architecture Overview

Figure 3 shows the processing pipeline. A proof is processed in a separate worker process with a wall-clock limit. The parser produces a proof model; global checks validate provenance and the dependency graph; a dispatcher then routes each step to the checker of its rule, with the ATP as fallback; and the verdict aggregation combines all issues into one of three verdicts.

Table 4: Non-functional requirements.

ID	Requirement
NFR1	**Soundness (fail-closed).** A proof is verified only if every step has been positively validated. Missing tools, tool errors, unparsable output, and unknown rules never lead to acceptance.
NFR2	**Time.** A proof must be processed within the 30-second wall-clock limit, including proofs with several thousand steps. The checker enforces its own limit and then abstains.
NFR3	**Robustness.** Malformed input, deeply nested formulas, and unexpected annotations must not crash the verifier or lead to an unjustified verdict.
NFR4	**Determinism.** The verdict does not depend on process scheduling or on the order of independent checks.
NFR5	**Diagnostics.** Every failure is traceable to a step and a reason (FR11).
NFR6	**Extensibility.** A new rule can be added without modifying the other checkers.
NFR7	**Testability.** Each checker is a pure function of the proof model and can be tested in isolation; the test suite reaches a coverage of at least 85 %.
NFR8	**Portability.** Python 3.12 and the parser library `lark` suffice for the core; an ATP is configured through environment variables and is optional.

5.3 Data Model

The parser turns each file into a list of annotated formulas. An *annotated formula* has a name, a role, a formula, and a justification, which is either a source (a `file` reference to the problem) or an *inference record*. An inference record consists of the rule name, a list of parent names, and a list of additional information items: the status, the `new_symbols` declaration, and, for Skolemization, the eliminated variable with its Skolem term. Formulas are represented as immutable abstract syntax trees with nodes for variables, constants, function terms, atoms, equalities, negations, binary connectives, n-ary conjunctions and disjunctions, and quantified formulas. The use of n-ary conjunctions and disjunctions makes flattening of nested connectives explicit and simplifies the junction and clausal checks. Roles, statuses, and rule names are enumerations, so that unknown values are detected at parse time or at dispatch time instead of silently ignored.

5.4 Global Checks

Three checks involve the whole proof and run before the step checks.

Provenance (FR2). Each `axiom` and `conjecture` step cites a problem file and a formula name. The checker looks up the formula in the problem file and compares it with the step's formula up to alpha-equivalence, i.e. up to consistent renaming of bound variables, and compares the roles. Alpha-equivalence is implemented as a simultaneous traversal of both trees with two mappings between the bound variables of the two formulas, so that no renaming of the trees is necessary. Before the traversal, consecutive quantifiers of the same kind are merged into one prefix and nested conjunctions and disjunctions are flattened.

Table 5: Traceability from requirements to modules and tests.

Requirement	Module	Tests
FR1	`parser/`	`test_parser`, `test_parser_transformer`, `test_ast_nodes`
FR2	`file_dir`, `alpha_eq`	`test_file_dir`, `test_alpha_equivalence`
FR3	`negated_conjecture_checker`	`test_negated_conjecture_checker`
FR4, FR5	`dependency_checker`	`test_dependency_checker`
FR6	`skolem_checker`	`test_skolem_checker`
FR7	`structural_`, `propositional_`, `junction_`, `clausal_`, `instantiate_`, `existential_gen_checker`	matching `test_*` files
FR8	`external_atp_checker`	`test_unchecked_inference_rules`
FR9–FR11	`main.py` (`check_proof_file`, `report_proof_check`)	`test_proof_checker_summary`, `test_main_atp_tristate`
NFR2	`main.py` (worker process), `deep_recursion`	timing measurements (Chapter 7)
NFR1, NFR3	all	`test_mutation_fuzzing`, `test_advanced_edge_cases`
NFR7	CI configuration	coverage gate in the CI workflow

Dependency graph (FR4, FR5). The graph with an edge from each step to its parents must be acyclic. Cycles are detected with a depth-first search with three colors; dangling references are reported by the step checks. In a second pass, no step except the negation step may cite a `conjecture`. Otherwise the proof would assume the statement it has to refute.

Termination (FR9). The derivation must contain a step with formula `$false` that depends, directly or indirectly, on the negated conjecture.

5.5 Step Dispatch and the Checker Interface

All step checkers have the same shape: they take the step and its parents and return a list of issues. An issue contains the step name and a reason. A checker never throws for invalid proofs; exceptions indicate defects of the verifier and lead to abstention. The dispatcher selects the checker by the rule name of the step and performs the checks common to all rules before calling it: the number of parents must match the rule's arity, every parent must exist, and the status must be the one the rule requires. Rule checkers therefore need to concentrate on the logical content.

Dispatch is implemented as a registry that maps rule names to checkers together with their arity and required status. Adding a rule means registering one entry and writing one function (NFR6).

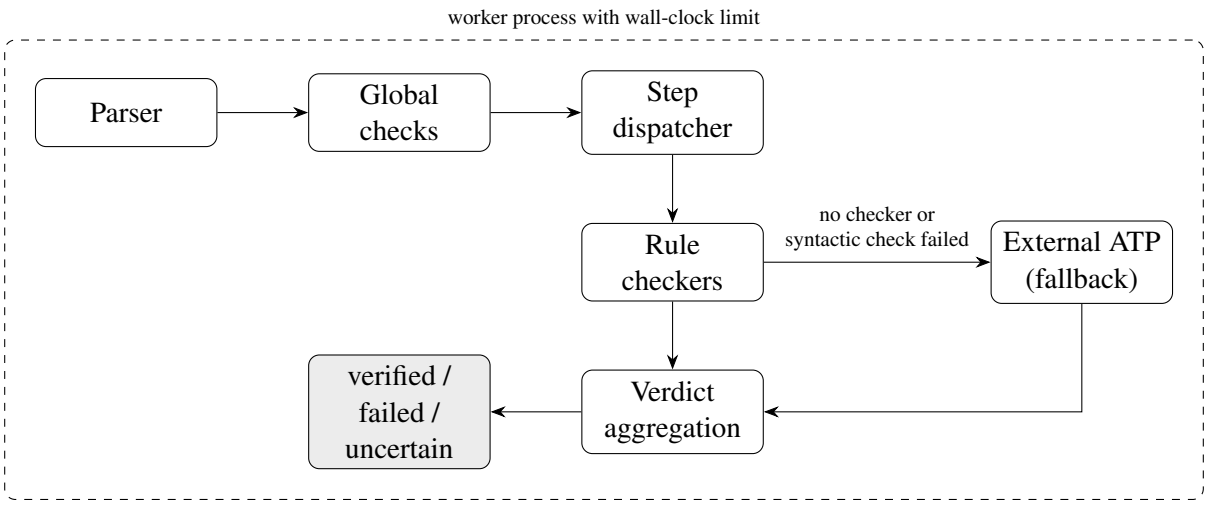


Figure 3: Architecture of the verifier. Parsing, checking, and aggregation run in a worker process with a wall-clock limit. [TODO: redraw in the diagrams folder, add issue flow if desired.]

5.6 Rule Checkers

Table 6 groups the supported rules into families. Most families reduce to alpha-equivalence and structural matching, which keeps them simple and fast.

5.7 Checking Skolemization

Skolemization cannot be validated by entailment (Section 2.3). The checker instead validates the following conditions, which together imply that the step is an instance of Skolemization:

1. The rule is `skolemize`, the step has exactly one parent, and its status is `esa`.
2. The step declares exactly one new symbol of kind `skolem`.
3. The Skolem term is a constant or a function applied to variables only, and these variables are exactly the universally quantified variables in whose scope the eliminated existential quantifier occurs.
4. Replacing the existential quantifier and its variable by the Skolem term in the parent formula yields the conclusion, up to alpha-equivalence.
5. The Skolem symbol is fresh: it neither occurs in the problem file nor was introduced by an earlier Skolemization step.

Conditions 3 and 5 are the side conditions that make the transformation equisatisfiability-preserving. Without the freshness condition, two steps could use the same symbol for different witnesses, which is unsound.

Table 6: Rule families and the principle of their checks.

Family	Rules	Principle
Structural	`copy`, `duplicate`, `rename_variable`, `double_negation`, `remove_double_negation`	Conclusion is alpha-equivalent to the (possibly negation-wrapped or unwrapped) parent.
Propositional	`modus_ponens`, `conjunction`, `split_conjunct`	Conclusion matches a conjunct of the parents or the consequent of an implication, also after universal instantiation.
Quantifier	`instantiate`, `existential_gen`	Template matching with abstract variables: conclusion is a substitution instance (resp. generalization) of the parent.
Junction	`weaken`, `commute`, `excluded_middle`	Shape check after flattening of conjunctions and disjunctions; ATP fallback for unusual variants.
Clausal / equational	`resolution`, `paramodulation`, `reflexivity`, `transitivity`, `rewrite`	Unification of literals or terms, construction of the expected resolvent or rewritten clause, comparison up to alpha-equivalence and quantifier permutation.
Negated conjecture	`negated_conjecture`	Conclusion is the negation of the parent conjecture, up to renaming of bound variables.
Skolemization	`skolemize`	See Section 5.7.
ATP-delegated	`consequence`, `add_hypothesis`, and every other rule with status `thm` or `cth`	Entailment from the step's premises by an external ATP (Section 5.8).

Polarity. Whether an existential quantifier may be eliminated, and which universal variables are in its scope, depends on its polarity. An existential quantifier in the antecedent of an implication or under a negation is effectively universal, so replacing it by a Skolem term would be unsound. The checker therefore tracks the polarity while it traverses the parent formula and rejects Skolemizations of quantifiers in negative position.

Herbrand symbols. Some `instantiate` steps carry a `new_symbols` annotation of kind `herbrand`. Instantiating a universal formula with an arbitrary term is entailed by that formula, so such a step is sound without a freshness condition and is validated like any other `thm` step.

5.8 Delegation to an External ATP

Steps that have no rule-specific checker are validated by an external ATP, in accordance with the competition rules. The same fallback applies to the junction rules if the syntactic check fails, to avoid false rejections of valid but unusual variants. The delegation obeys the following policy:

- **Eligibility.** Only steps with status `thm` or `cth` are delegated. Other statuses, in particular `esa`, are never delegated.

Table 7: Inference rules occurring in `PRV010+1.s` and their intended reading.

Rule	Intended reading
`excluded_middle`	introduces $\varphi \vee \neg\varphi$
`double_negation` / `remove_double_negation`	introduces / removes $\neg\neg\varphi$
`instantiate`	replaces a universal variable by a term
`existential_gen`	generalizes a term to an existential quantifier
`split_conjunct`	derives one conjunct from a conjunction
`reflexivity`	introduces $t = t$
`rename_variable`	renames a bound variable
`commute`	swaps the operands of a commutative connective
`weaken` / `add_hypothesis`	[TODO: verify; the code treats `add_hypothesis` as $\varphi \Rightarrow (H \Rightarrow \varphi)$ and
`skolemize`	replaces an existential variable by a fresh Skolem term
`consequence`	[TODO: verify; in the code, entailment of the conclusion from the listed

- **Isolation.** The ATP receives exactly the premises of the step as axioms and the step's formula as conjecture. It never sees the original conjecture or the rest of the proof.
- **Interpretation.** The answer is read from the SZS status line of the output. The statuses `Theorem`, `Unsatisfiable`, and `ContradictoryAxioms` confirm the step. Statuses such as `CounterSatisfiable` reject it.
- **Failure.** If the ATP is not found, times out, fails, or returns no SZS status, the step is *unresolved*. An unresolved step makes the verdict *uncertain*, never *verified*.
- **Failed syntactic check.** For rules with an ATP fallback (the junction rules), a failing syntactic check does not make the step invalid, since the check covers only the standard shapes of the rule. The step stays unresolved until the ATP has decided. For all other rules, a failing check makes the step invalid.
- **Budget.** An ATP call may not use more than the remaining wall-clock budget of the proof.

5.9 Verdicts and Soundness Argument

Each step ends in one of three states: *valid* (positively validated), *invalid* (a concrete defect was found), or *unresolved* (a required check could not be completed). The verdict of the proof is determined by the following rule, in this order:

1. If any step is invalid, or a global check fails, the verdict is *failed*.
2. Otherwise, if any step is unresolved, a timeout occurred, or an internal error occurred, the verdict is *uncertain*.
3. Otherwise the verdict is *verified*.

This implements the policy derived in Section 4.2. Table 8 relates the verdicts to the SZS status printed by the verifier and to the score.

Table 8: Verdicts, printed SZS status, and score under the ProoVer 2026 scheme.

Verdict	SZS output	Score if proof valid	Score if proof invalid
verified	`VerifiedGood`	$+1$	-10
failed	`VerifiedBad`	-1	$+2$
uncertain	`Timeout` [TODO: check which status is expected for giving up]	0	0

Soundness argument. Suppose the verdict is *verified*. Then (i) every axiom and conjecture is faithful to the problem, (ii) the negated conjecture is the negation of the conjecture, (iii) the dependency graph is acyclic, (iv) the derivation ends in `$false`, and (v) every other step is validated either by a rule check that shows entailment from the parents, by the Skolemization conditions, or by an ATP confirmation of entailment. Let Γ consist of the axioms and the negated conjecture, and let Γ_i be Γ together with the formulas of the first i steps in a topological order of the dependency graph. We show by induction that Γ_i is satisfiable whenever Γ is. The case $i = 0$ is trivial. If step i has status `thm`, its conclusion is entailed by its premises, which belong to Γ_{i-1}, so every model of Γ_{i-1} is a model of Γ_i. If step i is a Skolemization, its Skolem symbol is fresh, and its conclusion arises from the parent by replacing the existential variable by a Skolem term over the universal variables in scope; hence every model of Γ_{i-1} can be extended by a suitable interpretation of the Skolem symbol (a choice function) to a model of Γ_i. The last step derives `$false`, so Γ_n is unsatisfiable, and hence so is Γ. Since Γ consists of the axioms and the negation of the conjecture, Equation 1 shows that the conjecture follows from the axioms. The argument holds under the assumptions that the rule checkers are correct and that the ATP is sound; the thesis reduces the first by testing (Chapter 7) and makes the second explicit, because the competition rules require the use of an ATP.

5.10 Resource Control

The wall-clock limit is enforced by running each proof in a separate worker process that is terminated when the limit is exceeded. The verdict is then *uncertain*. Process isolation also contains crashes and memory exhaustion of a single proof. Formulas in the benchmark can be deeply nested; since all tree traversals are recursive, the worker runs in a thread with a 64 MB stack and a recursion limit of 100 000. The dependency analysis, which operates on the whole graph rather than on a formula, uses an iterative search to be independent of proof length.

5.11 Diagnostics

Every issue records the step name and a reason in plain language, for example that a copied formula does not match its parent. The report lists issues grouped by check and finishes with a summary line in the SZS format required by the competition. Because issues are data, the same information drives the console report, the summary table over many proofs, and the assertions in the tests.

Table 9: Main design decisions and the alternatives considered.

Decision	Alternative	Rationale
Syntactic checks first, ATP as fallback	ATP for every step	Faster and independent of an external tool for the vast majority of steps; the ATP is only needed for rare or unspecified rules.
Rule-local checker functions	One generic entailment check	Precise diagnostics and testable units; a generic check can neither explain failures nor handle `esa` steps.
Own AST with grammar-based parser	Reuse of an existing TPTP parser	Full control over annotations such as `new_symbols` and `skolemize` and over the AST with n-ary junctions; an LALR grammar with cached tables keeps parsing fast. [TODO: name the existing parsers considered and why they were not used].
Three-valued verdict	Binary verdict	Matches the scoring asymmetry (Section 4.2).
Process isolation per proof	In-process timeout	Reliable termination of non-cooperative code and containment of crashes.

5.12 Design Decisions and Alternatives

5.13 Summary

The design translates the requirements into a pipeline of parsing, global checks, rule-local step checks with a restricted ATP fallback, and a verdict rule that is derived from the competition's scoring. Its central property is that acceptance requires positive evidence for every step, which makes the soundness argument of Section 5.9 possible. Chapter 6 describes how the components are implemented in Python.

6 Implementation

This chapter describes how the design of Chapter 5 is realized in Python. Section 6.1 gives an overview of the code base. Sections 6.2 and 6.3 describe the data model and the parser, Section 6.4 the driver that dispatches the steps and computes the verdict. Sections 6.5 to 6.9 cover the checkers, Section 6.10 the resource control, and Sections 6.11 and 6.12 the test suite and the tooling used for the evaluation. Section 6.13 lists where the implementation falls short of the design.

6.1 Overview

The implementation is written in Python 3.12 and uses the parser library `lark` as its only runtime dependency. It consists of about 5 000 lines of code in four layers (Table 10): the parser and the AST, the rule checkers with shared utilities, the driver, and the tooling for scoring and performance measurement. The separation follows the responsibilities of the design: syntax handling is independent of logical verification, and each rule family lives in its own module, so that new constructs or

Table 10: Module structure of the implementation (approximate line counts).

Layer	Module (src/...)	Lines
Parser and AST	parser/parser.py, parser/tptp_grammar.lark, parser/ast_nodes.py	560
Constants	var_mapping.py (enumerations for roles, statuses, rules, connectives)	90
Global checks	checker/file_dir.py, dependency_checker.py, negated_conjecture_checker.py	500
Structural and propositional	structural_checker.py, propositional_checker.py, junction_checker.py	430
Quantifier rules	instantiate_checker.py, existential_gen_checker.py, skolem_checker.py	760
Clausal and equational	clausal_checker.py	470
Shared utilities	alpha_eq.py, formula_utils.py	400
ATP fallback	external_atp_checker.py	190
Driver	main.py, deep_recursion.py	1 130
Tooling	scoring/, performance/	650

rules can be added without touching the others.

6.2 Internal Representation

Formulas are represented by Python data classes defined in ast_nodes.py. Terms are Variable, Constant, and FunctionTerm; atomic formulas are Atom and Equality; compound formulas are Negation, BinaryFormula, JunctionFormula, and QuantifiedFormula. Conjunctions and disjunctions are JunctionFormula objects with a list of operands rather than nested binary trees, which simplifies the junction and clausal checks, since many rules operate on lists of literals. A QuantifiedFormula stores the quantifier and the list of bound variables, which the instantiation, generalization, and Skolemization checks rely on.

A proof step is an AnnotatedFormula with name, role, formula, and either a source or an InferenceRecord. The record contains the rule name, the parents, and a list of information items: the status, the new_symbols declaration (kind and symbols), and, for Skolemization, the skolemize information with the eliminated variable and the Skolem term. Roles, statuses, rule names, quantifiers, and connectives are StrEnum classes in var_mapping.py. This avoids inconsistencies caused by ad hoc string handling and makes the set of recognized values explicit.

6.3 Parser

The parser is generated by Lark from the grammar tptp_grammar.lark using the LALR algorithm with cached tables. The parser object is created once per process through a cached helper. A TPTPTransformer maps the grammar productions to the data classes above. During this mapping some constructs are normalized: for instance, the connective <= is converted into a canonical implication direction, so that later checks have fewer cases. The parser also handles source

annotations, inference records, and `include` directives. The function `load_proof` combines parsing with the extraction of the referenced problem and returns a `ProofFile` object.

Because the grammar is recursive, deeply nested formulas lead to deep recursion in the transformer and in all checkers. Section 6.10 describes how this is handled.

6.4 Driver and Dispatch

The driver is the function `check_proof_file` in `main.py`. It parses the problem file and the proof, builds an index from step names to steps, runs the global checks, and then iterates over the steps in file order. For each step it selects a checker by the step's role and rule name in an `if/elif` chain: axioms and conjectures go to the provenance check, the negated conjecture to its dedicated checker, the rules of Table 6 to their modules, `skolemize` to the Skolem checker, and every other step with status `thm` or `cth` to the external ATP. Steps with another status and no checker, and steps without any justification, are reported as having no checker.

The driver does not stop at the first error. Each check writes its findings to a result map keyed by step name; there is one map per rule family (for example `skolem_issues` or `resolution_issues`), one for the dependency checks, and one for the ATP fallback. The maps are returned to the caller, so that the console report, the summary table, and the tests all work on the same data (Section 5.11). The driver keeps two sets for the freshness check of Skolem symbols: the function symbols of the problem file, collected once, and the Skolem symbols introduced so far. A `skolemize` step is rejected if its new symbol is in one of these sets.

Verdict. The checker `check_proof_file` returns the issue maps together with the flag `all_ok`, which is true if all maps are empty. The reporting function maps this flag to the SZS status `VerifiedGood` or `VerifiedBad`; a timeout yields `Timeout`.

6.5 Global Checks

The provenance check in `file_dir.py` verifies that an imported axiom or conjecture carries a `file(...)` annotation, that the cited file and formula name exist in the problem, that the role matches, and that the formula is alpha-equivalent to the original. Alpha-equivalence is implemented in `alpha_eq.py` as a simultaneous traversal with two mappings between bound variables. Before the traversal, consecutive quantifiers of the same kind are merged into one prefix and nested junctions of the same connective are flattened (Figure 4). The comparison is used for provenance, for the structural rules, and for many quantifier rules.

The dependency checks in `dependency_checker.py` detect cycles with an iterative depth-first search with three colors and report the cycle; dangling parents are ignored there and reported by the step checks. A second function rejects any step other than the negation step that cites a `conjecture`. The negated conjecture checker verifies role, status `cth`, the parent, and that the formula is the negation of the conjecture, up to renaming of bound variables.

6.6 Structural, Propositional, and Quantifier Rules

The structural rules (`copy`, `duplicate`, `rename_variable`, `double_negation`, `remove_double_negation`) are checked in `structural_checker.py`: the checker verifies the metadata and that the conclu-

```
def is_alpha_equivalent(f1, f2):
    return _alpha_eq(normalize(f1), normalize(f2), {}, {})

def _alpha_eq(a, b, map1, map2):
    if type(a) is not type(b): return False
    if a is a Variable:                        # bound or free
        if a in map1 or b in map2:             # bound: mappings must agree
            return map1.get(a) == b and map2.get(b) == a
        return a == b                          # free: names must be equal
    if a is a Quantified formula:
        require same quantifier and arity
        extend map1, map2 with the bound variables of a and b
        result = _alpha_eq(a.body, b.body, map1, map2)
        restore map1, map2                     # scoping
        return result
    otherwise: compare symbols, then recurse over the children in order
```

Figure 4: Structure of the alpha-equivalence check (pseudocode, simplified).

sion has the expected relation to its parent, mostly by alpha-equivalence. The propositional rules (`modus_ponens`, `conjunction`, `split_conjunct`) and the junction rules (`weaken`, `commute`, `excluded_middle`) act on the top-level connective structure, where the explicit junction nodes pay off. The syntactic checks for `weaken`, `commute`, and `excluded_middle` cover only the standard shapes of the rules. If such a check fails and the step has status `thm`, the driver asks the external ATP before it reports a failure, so that valid but unusual variants are not rejected.

The `instantiate` and `existential_gen` checks use template matching (`match_template_to_instance` in `formula_utils.py`): the quantified parent is treated as a template with abstract variables, and the checker searches for a consistent substitution that turns it into the conclusion, or the other way around for generalization. Shared utilities for flattening quantifier prefixes, collecting function symbols, and applying substitutions are collected in `formula_utils.py`, so that all checkers interpret formulas consistently.

6.7 Skolemization

The Skolem checker (`skolem_checker.py`, the largest checker) implements the conditions of Section 5.7. It first validates the shape of the step: rule `skolemize`, exactly one parent, exactly one status equal to `esa`, exactly one `new_symbols` declaration of kind `skolem` with one symbol, and a `skolemize` information item that names the eliminated variable. It then locates the quantifier that binds the eliminated variable in the parent, analyzing its syntactic and its effective quantifier kind while tracking the polarity (negations and the antecedent of implications flip it), and computes the universal variables in whose scope it occurs. Finally it checks that the Skolem term is built from the declared symbol and exactly these variables, and that applying the replacement to the parent yields the conclusion up to alpha-equivalence; for this comparison, bound variables are renamed apart first to avoid capture. The freshness of the symbol is not decided in the checker itself but in the driver,

```
validate_step_with_atp(step, premises, timeout) -> True | False | "unknown"
    write premises as axioms, step as conjecture to a temporary .p file
    run prover (subprocess, timeout)
    executable not found        -> "unknown"
    timeout                     -> "unknown"
    no "% SZS status" line      -> "unknown"
    Theorem | Unsatisfiable | ContradictoryAxioms -> True
    any other status            -> False
```

Figure 5: Result mapping of the ATP fallback.

which holds the symbol sets across steps (Section 6.4).

6.8 Equality and Clausal Reasoning

The module `clausal_checker.py` implements `resolution`, `paramodulation`, `reflexivity`, `transitivity`, and `rewrite`. Resolution decomposes the parents into lists of disjuncts, searches for complementary literals, computes unifiers where necessary, builds the resolvent, and compares it to the stated conclusion. Quantified clauses are handled by flattening the leading universal prefix and rebuilding it for the conclusion. Paramodulation rewrites occurrences of one side of an equation from one parent in the other parent and compares the result with the conclusion; bound variables are renamed apart before rewriting to avoid variable capture. Reflexivity checks for a trivial equality $t = t$, and transitivity checks that two parent equalities chain to the conclusion.

The `rewrite` rule is the weakest check. In the benchmark format, rewrite steps may omit the demodulating equation, so the intended transformation cannot be reconstructed fully, and the checker verifies only the limited behavior observed in the benchmarks.

6.9 Delegation to the External ATP

The module `external_atp_checker.py` implements the fallback of Section 5.8. The function `status_allows_atp` implements the eligibility rule: only the statuses `thm` and `cth` may be delegated. The function `validate_step_with_atp` serializes the premises as axioms and the step as conjecture into a TPTP problem (with quoting of symbols and variable names that are not valid TPTP tokens), writes it to a temporary file, and starts the prover as a subprocess with a time limit. The default prover is Vampire in CASC mode; the command can be configured through the environment variables `EXTERNAL_ATP_COMMAND` and `EXTERNAL_ATP_BINARY`, and the module translates file paths if the prover is launched through WSL. The answer is read from the SZS status line of the output (Figure 5).

The function returns a tri-state value, so that the caller can distinguish a refutation from an unresolved call. The prover receives only the parents of the step, never the original conjecture or the rest of the proof. The calls are sequential, and each call may use the full per-proof time limit.

6.10 Resource Control

Each proof is checked in a separate process created with Python's `multiprocessing` module. The parent waits for the result for the configured number of seconds (30 by default, configurable on the command line) and terminates the worker if the limit is exceeded. The worker passes its result back through a queue; exceptions are caught and returned as an error message. Process isolation contains crashes and memory exhaustion of a single proof and allows non-cooperative code, such as a hanging ATP call, to be terminated reliably.

Inside the worker, the checking runs in a dedicated thread with a stack of 64 MB and a recursion limit of 100 000 (`deep_recursion.py`). This is necessary because both the parser's transformer and the checkers traverse formulas recursively; the benchmark contains stress tests with hundreds of nested negations or function applications. Raising only the recursion limit would risk a hard interpreter crash, because each Python frame also needs space on the operating system's thread stack. The dependency analysis uses an iterative search and is therefore independent of proof length.

6.11 Test Suite

The test suite consists of about 250 `pytest` test functions in 21 files. Most files exercise one module in isolation, for example `test_skolem_checker` or `test_clausal_checker`, which is possible because the checkers are functions of the step and its parents. In addition, there are tests for the parser and the AST, a parametrized provenance test over the example proofs (`test_all_examples`), tests for the tri-state ATP result (`test_external_atp_checker`, `test_main_atp_tristate`), edge-case tests, and a mutation-based test (`test_mutation_fuzzing`) that perturbs seed proofs and checks that the verifier reacts with the expected outcome or issue category. A GitHub Actions workflow runs the suite on Python 3.12 for every push and pull request and fails if the line coverage is below 85 % (NFR7).

6.12 Tooling for Reporting and Evaluation

The command-line interface in `main.py` checks either a single proof or, by default, all proofs of the `ProoVer2026` directory. For each proof it prints a report grouped by check and finishes with an SZS line, and for a batch it prints counts of the three outcomes and a per-proof summary table with the failed checks. The scoring module (`src/scoring`) compares the outcomes with expected results given in CSV files and computes the points of the ProoVer scheme ($+2$, $+1$, 0, -1, -10) per proof and in total, together with a breakdown of errors by check category. The performance module (`src/performance`) records wall-clock and CPU time per operation category and writes them to a CSV file. These two modules produce the numbers reported in Chapter 7.

6.13 Deviations from the Design

The implementation does not yet realize all aspects of the design: the dispatcher is an `if/elif` chain instead of a registry; the verdict is derived from a Boolean flag instead of the three values of Section 5.9, so that unresolved ATP calls are reported like failures; the global check that the derivation ends in `$false` (FR9) is not implemented; `new_symbols` annotations of kind `herbrand`

Table 11: Groups of the TST set.

Group	Proofs	Purpose
`ax`	11	Provenance: modified, renamed, or re-roled axioms and conjectures, missing `file` directive
`c`	13	Correct proofs with edge cases: Skolemization, renamed steps, unordered steps, nested quantifiers
`cp`	3	Conjecture used as premise, inconsistent axioms
`nc`	11	Negated conjecture: wrong status, parent, rule, role, or formula; missing or duplicated
`sk`	25	Skolemization: arity, scope, freshness, polarity, wrong result, wrong number of parents or symbols
`st`	13	Structure: no `$false`, invalid `thm` step, cycles, undefined premises, unknown status

are parsed but not checked separately; and the `rewrite` rule is checked only for the shapes observed in the benchmark.

7 Evaluation

This chapter evaluates the verifier against the research questions of Section 1.3. Section 7.1 describes the benchmarks, the metrics, and the experimental setup. Section 7.2 reports correctness and score, Section 7.3 the coverage of the inference rules (RQ1), and Section 7.4 the properties of the architecture (RQ2). Sections 7.5 to 7.7 report run time, robustness, and ablations (RQ3), and Sections 7.8 and 7.9 analyze the remaining errors and the threats to validity.

7.1 Setup

Benchmarks. The evaluation uses two proof sets. The *ProoVer set* consists of the 100 proofs `PRV000+1.s` to `PRV099+1.s` of the ProoVer 2026 benchmark, of which 47 are valid and 53 contain a deliberate error. The *TST set* consists of 76 proofs written for this thesis to test individual checks. It is organized in six groups (Table 11); each proof has an expected result of *correct* or *incorrect*, and one proof is marked as an edge case whose expected result is open (inconsistent axioms, `cp03`). The expected results of the ProoVer set were [taken from the organizers / determined by manual analysis; TODO: state the source].

Metrics. The primary metric is the total score under the ProoVer 2026 scheme (Section 4.2): $+1$ for a correctly accepted valid proof, $+2$ for a correctly rejected invalid proof, 0 for abstention, -1 for a wrongly rejected valid proof, and -10 for a wrongly accepted invalid proof. In addition we report the verdict matrix (Table 12), the number of wrong acceptances separately, the run time per proof (wall-clock and CPU), the share of steps that are decided internally, and the peak memory.

Environment. All measurements were made on [CPU, number of cores, RAM, operating system] with Python [3.12.x], `lark` [version], and Vampire [version] as external prover, with the time limit

Table 12: Verdicts on the ProoVer set (number of proofs) and resulting score.

Expected \ verdict	verified	failed	uncertain / timeout	Points
valid (47)	[ ]	[ ]	[ ]	[ ]
invalid (53)	[ ]	[ ]	[ ]	[ ]
Total	[ ]	[ ]	[ ]	[S]

Table 13: Results on the TST set by group (agreeing / total).

	`ax`	`c`	`cp`	`nc`	`sk`	`st`	Total
Verdict as expected	[ ]/11	[ ]/13	[ ]/2	[ ]/11	[ ]/25	[ ]/13	[ ]/75

of 30 s per proof. Each proof was checked in a fresh worker process. Run times are medians of [5] repetitions after one warm-up run; the first run of the Lark parser builds its cached tables. The exact commit of the verifier is [hash].

7.2 Correctness and Score

Table 12 shows the verdicts of the verifier against the expected results. The verifier accepts [X] of the 47 valid proofs of the ProoVer set, rejects [Y] of the 53 invalid proofs, abstains on [Z] proofs (of which [T] are timeouts), and wrongly accepts [W] proofs. This results in a total score of [S] of a maximum of $47 + 2 \cdot 53 = 153$ points.

The central safety property is that no invalid proof is accepted. For the ProoVer set this holds ([confirm]); the same holds for the TST set, where the verdicts agree with the expectation in [A] of 75 proofs. Table 13 breaks the TST results down by group. Wrong rejections of valid proofs cost only one point each; we list them in Section 7.8, because they point at gaps in rule coverage rather than at unsoundness.

7.3 Rule Coverage (RQ1)

Table 14 lists, for each inference rule occurring in the ProoVer set, how often it occurs and how its steps were decided: by the internal checker, by the ATP after a failed syntactic check, by the ATP because no internal checker exists, or not at all. Together with the verdicts, this shows which rules a verifier must support to cover the benchmark (RQ1) and how much of the proof checking is independent of the external prover. The driver already counts the branch taken for each step in its debug output (`DEBUG_PROOF_FLOW`); these counters are the source of the table.

7.4 Architecture Metrics (RQ2)

To assess whether the architecture keeps the trusted part small and extensible, we report three measures. First, the size of each checker in lines of code and its statement coverage by the unit tests (Table 15); the checkers range from about 70 lines (`existential_gen_checker`) to about 590 lines (`skolem_checker`), and the whole checker package has about 2 700 lines. Second, the share of steps that are decided without the external prover (Section 7.3), which determines how much of the

Table 14: Rules in the ProoVer set and how their steps were decided.

Rule	Steps	Internal	ATP fallback	ATP only	Unresolved
`skolemize`	[ ]	[ ]	–	–	[ ]
`instantiate`	[ ]	[ ]	–	–	[ ]
`resolution`	[ ]	[ ]	–	–	[ ]
`weaken / commute`	[ ]	[ ]	[ ]	–	[ ]
`consequence`	[ ]	–	–	[ ]	[ ]
...					
Total	[ ]	[ ] ([%])	[ ]	[ ]	[ ]

Table 15: Size and test coverage of the checker modules.

Module	Lines	Coverage
`skolem_checker`	[ ]	[ ] %
`clausal_checker`	[ ]	[ ] %
...		
Checker package	[ ]	[ ] %

verdict depends on the prover. Third, the effort to add a rule, which we demonstrate with a worked example: [TODO: choose a rule that was added late, e.g. `add_hypothesis` or `excluded_middle`, and report the number of changed lines and the files touched; with the registry of Section 5.5 the answer is one entry and one function].

7.5 Run Time and Memory (RQ3)

Figure 6 plots the wall-clock time of each proof of the ProoVer set against its number of steps, with the ATP time shown separately. We report the median, the 90th percentile, and the maximum, the number of proofs above 10 s, and the number of timeouts. Preliminary measurements with five proofs per category (Table 16) already show the expected pattern: the internal checks take milliseconds to a few hundred milliseconds, while a single call of the external prover takes more than two seconds on average and thereby dominates the run time of every proof that needs it.

We also report where the time goes for the slowest proofs: parsing (LALR parser and transformer), global checks, rule checks, and ATP calls, and the peak memory of the worker process ([MB], measured with [tool]). Deeply nested stress tests such as `PRV042` show whether the larger stack of Section 6.10 suffices: [result].

7.6 Robustness (RQ3)

Robustness is evaluated in three ways. First, the mutation test generates variants of valid seed proofs by applying one mutation each, such as changing a connective, swapping quantifiers, renaming a Skolem symbol to an existing one, or deleting a premise (Table 17); a mutant is *killed* if the verifier does not accept it. Second, syntactically broken inputs (truncated files, unknown rules, missing annotations, deeply nested formulas) must neither crash the verifier nor lead to acceptance;

Table 16: Preliminary average run time per check category (five proofs each; to be repeated on the final version and the final hardware).

Category	Wall-clock (s)	CPU (s)
Axiom verification	0.0021	0.0011
Copy	0.0049	0.0038
Double negation	0.0108	0.0103
Weaken	0.0362	0.0306
Conjunction	0.0807	0.0570
Modus ponens	0.1491	0.1259
Instantiate	0.2207	0.1751
Resolution	0.3272	0.2874
Skolemization check	0.4647	0.3799
External ATP call	2.2303	1.8532

[Placeholder: wall-clock time per proof against the number of steps]

Figure 6: Wall-clock time per proof of the ProoVer set against the number of steps.

we report the outcome for each class. Third, we repeat the ProoVer run with the external prover removed from the path. With the fail-closed policy, the number of wrong acceptances must stay at zero; the number of verified proofs drops by the proofs that need the ATP, and these proofs become uncertain. This experiment verifies NFR1 empirically.

7.7 Ablations

Two ablations quantify the design decisions of Table 9. *Syntactic checks first:* we run the verifier with all rule checkers disabled, so that every step goes to the external prover, and compare score and run time. This shows what the internal checkers buy: [time factor] and [difference in score]. *Three-valued verdict:* we replace the verdict rule by a binary one, in which unresolved steps count as failures, and recompute the score; the difference [D] shows the value of abstaining as described in Section 4.2.

7.8 Error Analysis

We classify every proof of the ProoVer set whose verdict differs from the best possible one (acceptance of a valid proof, rejection of an invalid one) by its cause. Table 18 counts them by category: unsupported or imprecisely checked rule, missing ATP result, timeout, parser limitation, and expected result uncertain.

7.9 Threats to Validity

Internal validity. The verifier was developed against the ProoVer 2026 sample proofs, so the ProoVer set is not an independent test set; the TST set was written by the author and reflects the author's understanding of the rules. The expected results of the ProoVer set depend on [source]. We

Table 17: Mutation operators and kill rates.

Mutation operator	Mutants	Killed	Kill rate
Change connective	[ ]	[ ]	[ ] %
Swap quantifier kind	[ ]	[ ]	[ ] %
Reuse existing symbol as Skolem symbol	[ ]	[ ]	[ ] %
Remove premise	[ ]	[ ]	[ ] %
…			
Total	[ ]	[ ]	[ ] %

Table 18: Causes of suboptimal verdicts on the ProoVer set.

Cause	Proofs	Example
Rule not checked precisely (e.g. `rewrite`)	[ ]	[ ]
ATP unresolved (timeout, no status)	[ ]	[ ]
Time limit exceeded	[ ]	[ ]
Parser or normalization limitation	[ ]	[ ]

mitigate this by reporting the mutation experiments, which do not depend on the benchmark, and by separating the wrong acceptances from the other errors.

External validity. The benchmark contains FOF proofs from a fixed set of generators; the results do not transfer to proofs from other provers with different rule vocabularies or to other logics. The scoring scheme and the 30 s limit are those of one competition.

Construct validity. The correctness of the accepted proofs relies on the correctness of the rule checkers, which is supported by tests but not proved, and on the soundness of the external prover. Run times depend on the hardware and on the prover version and are therefore reported with their environment and as medians over several runs.

7.10 Summary

[TODO: three to five sentences answering RQ1–RQ3 with the key numbers; the full answers follow in Chapter 9.]

8 Discussion

This chapter interprets the results of Chapter 7, discusses what the verifier does and does not guarantee, and lists its limitations.

8.1 Interpretation of the Results

The verifier reaches a score of [S] of 153 points on the ProoVer set and does not accept any invalid proof [confirm]. The second property matters more than the first: with a penalty of -10 for a wrong acceptance, a single such error costs as much as ten correct acceptances. The verdict matrix shows where the remaining points are lost. [Interpretation: most losses are abstentions on valid proofs, caused by [rule / timeout / ATP], and not wrong rejections.] This is the intended behavior of the fail-closed policy of Section 4.2: when the verifier loses points, it loses them in the cheap direction.

The ablation of the verdict rule (Section 7.7) quantifies this: a binary verdict that treats unresolved steps as failures would have changed the score by [D] points, because [it turns abstentions on invalid proofs into lucky rejections but also turns abstentions on valid proofs into wrong rejections]. The comparison also shows that the benefit of abstaining depends on the benchmark: it pays off when unresolved steps are rare but dangerous, and it is worth little when almost every proof can be decided.

The rule coverage (Section 7.3) shows that [X %] of the steps are decided by internal checks, and only [Y %] need the external prover. At the same time, the run times (Section 7.5) are dominated by the ATP calls: a single call takes about two seconds in our preliminary measurements, while all internal checks of a proof together take [milliseconds to a few hundred milliseconds]. For this reason, the order "cheap checks first" is not only a design preference but the main lever for the run time, and it is the reason why the verifier stays within the 30 s limit on [Z] of the proofs.

8.2 Scoring Policy and Its Limits

The policy of Section 4.2 follows from the scoring scheme, but it rests on assumptions that deserve a comment. The probability p that a proof is valid is not observable, so the thresholds $2/3$ and $10/11$ cannot be applied literally; the verifier approximates them by the discrete rule that acceptance requires positive evidence for every step and rejection requires a concrete defect. This is conservative on both sides. It would be possible to accept proofs with a small unresolved remainder if the remainder is judged harmless, but this requires a calibrated estimate of p that the verifier does not have. The policy is also tied to this competition: with a different penalty the optimal behavior changes, and in a setting where a wrong acceptance is not much worse than a wrong rejection, abstention would be less attractive. Finally, the policy trades points for safety when the external prover is missing or too slow: invalid proofs whose defect is only visible to the prover become *uncertain* instead of being rejected, and the verifier forgoes the $+2$ for them.

8.3 Trusted Computing Base and Soundness

The soundness argument of Section 5.9 is conditional: it holds if the rule checkers are correct and the external prover is sound. In terms of the trusted computing base (Section 2.1), the following components therefore have to be trusted for the property that every accepted proof is valid:

- the *parser and normalization*, since a parse error that changes the meaning of a formula would change what is checked;
- the *alpha-equivalence* check, which also merges quantifier prefixes and flattens junctions before it compares, and which therefore equates formulas that are not literally identical;

- the *rule checkers*, in particular the Skolem checker, the largest module, and the clausal checker;
- the *serialization of formulas to TPTP* for the external prover, because a defect there would make the prover validate a different formula than the one in the proof;
- the *external prover* and its interpretation through the SZS status line; and
- the *verdict aggregation*, which decides how the individual results are combined.

Compared with a checker that sends every step to a prover, this list is longer, because the internal checks are part of it; compared with the prover that produced the proof, it is much shorter. This is the trade-off stated in Section 3.5: the thesis exchanges a larger trusted checker for less dependence on the prover and for better diagnostics. The evaluation supports the correctness of the checkers by tests, by the mutation experiment, and by the benchmark, but it does not prove it. A way to reduce the trusted part further would be to restrict the internal checks to the rules where they are simple (structural, propositional, and the Skolem conditions) and to delegate the rest, or to validate the checks themselves against a proof assistant.

8.4 Design Trade-offs

Internal checks and ATP fallback. The competition rules allow any step except Skolemization and the import steps to be checked by an external prover. A single step that derives `$false` directly from the axioms and the negated conjecture is then delegated completely (the test group `st08` contains such a "monolithic" step). For such proofs the verifier is exactly as strong as the prover, and the internal checks contribute only the global conditions. This is correct with respect to the rules, but it shows that the quality of the verifier on such inputs lies in the global checks, not in the step checks.

Modularity. Rule-local checkers made it possible to test each rule in isolation and to attribute every error to a step and a reason. The price is that some knowledge, such as the freshness of Skolem symbols, cannot be local to one step and lives in the driver. The size of the dispatcher in `main.py` is a sign that the modular idea has been applied to the checkers but not yet to the dispatch; the registry of Section 5.5 addresses this.

Python. The implementation language favors readability and fast development, which was valuable for a thesis with many rules. It costs run time for the internal checks and complicates deep recursion, which needed a dedicated thread with a larger stack. The measurements show that the ATP, not Python, dominates the run time of the proofs that need it; the pure Python cost matters for the large proofs, where [parsing / checks] take [x s].

Process isolation. Running each proof in its own process makes timeouts and crashes harmless and enforces the time limit, but it adds a startup cost per proof and makes sharing state, such as a cache of ATP results, harder.

8.5 Limitations

- **Logic and format.** The verifier handles FOF problems and TSTP proofs with the rule vocabulary of the ProoVer 2026 benchmarks. CNF, typed, and higher-order input are out of scope, and rules outside the supported set are delegated to the prover or lead to abstention.
- **Weak rule checks.** The `rewrite` rule is checked only for the shapes observed in the benchmark, because the demodulating equation may be missing in the proof. Since this check is weaker than the logical meaning of the rule, an accepted `rewrite` step is less well justified than the other steps.
- **Symbols from instantiation.** Annotations of Herbrand symbols are parsed but not checked separately. This is sound for `thm` steps, since instantiation with an arbitrary term is entailed by the parent, but it means that the annotation carries no checked information.
- **Sequential prover calls.** Each unresolved step starts its own prover process, one after another. Proofs with many such steps are therefore slow. Parallel calls and a shared time budget would help.
- **Benchmark.** The ProoVer set contains 100 proofs and the TST set 76 proofs written by the author. This is enough to find defects, but not to estimate rare failure rates, and the results do not carry over to proofs from other provers (Section 7.9).
- **Trust in tests.** Correctness of the checkers is supported by tests and mutations, not proved.

9 Conclusion and Future Work

9.1 Summary

This thesis designed and implemented a Python-based verifier for TSTP derivations for the ProoVer 2026 competition. The starting point was the observation that the competition's scoring scheme is highly asymmetric: a wrongly accepted proof costs ten points, a correct acceptance gains one. From the expected score of the three possible answers, we derived a fail-closed policy with a three-valued verdict: the verifier accepts only if every step is validated positively, rejects only if it has found a concrete defect, and abstains otherwise. The verifier realizes this policy as a modular pipeline: a parser, global checks of provenance, dependencies, and the negated conjecture, rule-local checkers for structural, propositional, quantifier, and clausal rules, a dedicated check for Skolemization including the freshness of the introduced symbols, and a restricted fallback to an external prover for the remaining steps. A soundness argument shows that acceptance implies unsatisfiability of the axioms together with the negated conjecture, under the assumption that the rule checkers are correct and the prover is sound.

9.2 Answers to the Research Questions

RQ1: Which rules and fragments must be supported? The ProoVer 2026 benchmarks use [N] distinct inference rules. Of these, [A] are checked internally, which covers [X %] of all steps;

the remaining [B] rules, among them `consequence`, are delegated to the prover. Skolemization is the one rule that has to be checked internally, since it preserves only satisfiability and cannot be validated by entailment, and the checks for quantifier scope, polarity, and freshness are needed to reject the subtle errors in the invalid proofs.

RQ2: How does modularity keep the trusted part small and extensible? Every rule check is a function of a step and its parents and returns structured issues. The checkers can be tested in isolation, which is reflected in a test suite of about 250 tests with a coverage of [C %], and an error can be attributed to a step and a reason. The trusted part consists of the parser, the alpha-equivalence check, the rule checkers, the serialization to the prover, and the verdict aggregation, in total about [L] lines, plus the external prover as an assumption (Section 8.3). [Extending the verifier by a rule required [k] lines in [m] files.]

RQ3: How do correctness, run time, and memory behave? On the ProoVer set, the verifier reaches [S] of 153 points with [W] wrong acceptances, a median run time of [t s], and a maximum of [t_{max} s]; [T] proofs hit the time limit. The mutation experiment kills [M %] of [N] mutants, and with the prover removed the verifier still makes no wrong acceptance, which confirms the fail-closed property empirically. Run time is dominated by the calls to the external prover, not by the internal checks.

9.3 Contributions Revisited

The thesis contributes (i) a verdict policy for proof checking derived from the scoring scheme, together with a soundness argument that accounts for the different semantics of `thm` and `esa` steps; (ii) a modular architecture and its implementation with a dedicated Skolem check and a restricted ATP fallback; and (iii) an evaluation on the competition benchmark and on a suite of mutated and hand-written proofs that measures correctness, coverage, run time, and robustness, with an analysis of the remaining limitations.

9.4 Future Work

Several directions follow from the limitations of Section 8.5.

- **Registry-based dispatch.** Replace the `if`/`elif` dispatcher by the registry of the design, so that adding a rule needs one entry and one function.
- **Stronger checks for weak rules.** Reconstruct the missing demodulating equation of `rewrite` steps, or validate them against a prover, and add checks for Herbrand symbols once their meaning is fixed by the specification.
- **Counter-model search.** Use a model finder to reject steps that are not entailed, as Nörgler does, which would turn more unresolved steps into defects and thereby recover the $+2$ for invalid proofs.
- **Parallel and budgeted prover calls.** Run the prover calls for independent steps in parallel under a shared time budget, and cache results across steps.

- **Smaller trusted base.** Validate the internal checks themselves, for example by formalizing the Skolem conditions in a proof assistant, or produce certificates that a small kernel can check, following the approaches discussed in Chapter 3.
- **Broader input.** Support CNF proofs and further rule vocabularies, and compare the verifier with GDV and Nörgler on a common benchmark.

10 Notes on form

10.1 Formatting

This LaTeX template uses the following formatting:

- font: Linux Libertine O (alternatively: Times New Roman)
- font size: 12 pt
- left and right margin: 3.5 cm, top and bottom margin: 3 cm
- align: left
- line spacing: one and a half (alternative: 15 pt line spacing with 12 pt font size)

When implementing the specifications in Word, it is essential to define style sheets.

10.2 Citation

The citation method follows the author-year system. Place reference is in the text, footnotes should only be used for explanations and comments. The following notes are taken from the *language* bibliography template from `ron.artstein.org`:

The *Language* style sheet makes a distinction between two kinds of in-text citations: citing a work and citing an author.

- Citing a work:
  - Two authors are joined by an ampersand (&).
  - More than two authors are abbreviated with *et al.*
  - No parentheses are placed around the year (though parentheses may contain the whole citation).
- Citing an author:
  - Two authors are joined by *and*.
  - More than two authors are abbreviated with *and colleagues*.
  - The year is surrounded by parentheses (with page numbers, if present).

To provide for both kinds of citations, `language.bst` capitalizes on the fact that `natbib` citation commands come in two flavors. In a typical style compatible with `natbib`, ordinary commands such as `\citet` and `\citep` produce short citations abbreviated with *et al.*, whereas starred commands such as `\citet*` and `\citep*` produce a citation with a full author list. Since *Language* does not require citations with full authors, the style `language.bst` repurposes the starred commands to be used for citing the author. The following table shows how the `natbib` citation commands work with `language.bst`.

Command	Two authors	More than two authors
`\citet`	Hale & White Eagle (1980)	Sprouse et al. (2011)
`\citet*`	Hale und White Eagle (1980)	Sprouse and colleagues (2011)
`\citep`	(Hale & White Eagle 1980)	(Sprouse et al. 2011)
`\citep*`	(Hale und White Eagle 1980)	(Sprouse and colleagues 2011)
`\citealt`	Hale & White Eagle 1980	Sprouse et al. 2011
`\citealt*`	Hale und White Eagle 1980	Sprouse and colleagues 2011
`\citealp`	Hale & White Eagle 1980	Sprouse et al. 2011
`\citealp*`	Hale und White Eagle 1980	Sprouse and colleagues 2011
`\citeauthor`	Hale & White Eagle	Sprouse et al.
`\citeauthor*`	Hale und White Eagle	Sprouse and colleagues
`\citefullauthor`	Hale und White Eagle	Sprouse and colleagues

Authors of *Language* articles would typically use `\citet*`, `\citep`, `\citealt` and `\citeauthor*`, though they could use any of the above commands. There is no command for giving a full list of authors.

10.3 Bibliography

The bibliography of this template includes the references of the *language* stylesheet as a sample bibliography.

11 General Addenda

If there are several additions you want to add, but they do not fit into the thesis itself, they belong here.

11.1 Detailed Addition

Even sections are possible, but usually only used for several elements in, e.g. tables, images, etc.

12 Figures

12.1 Example 1

12.2 Example 2

List of Figures

1 The ProoVer 2026 proof `PRV000+1.s` (comments and SZS delimiters omitted). Axiom and conjecture are imported from the problem file, `neg` negates the conjecture, and one `consequence` step derives `$false` from `neg` and `a1`. 5
2 Expected score of accepting, rejecting, and abstaining as a function of the probability p that the proof is valid, under the ProoVer 2026 scoring scheme. Accepting beats abstaining only for $p > 10/11$, rejecting only for $p < 2/3$. 12
3 Architecture of the verifier. Parsing, checking, and aggregation run in a worker process with a wall-clock limit. [TODO: redraw in the diagrams folder, add issue flow if desired.] 16
4 Structure of the alpha-equivalence check (pseudocode, simplified). 23
5 Result mapping of the ATP fallback. 24
6 Wall-clock time per proof of the ProoVer set against the number of steps. 29

List of Tables

1 Systems entered in ProoVer 2026, as described in their system descriptions. "Ext." lists external tools the system relies on at run time. 9
2 Comparison of the closest related systems with this thesis. [TODO: fill in the last column according to the implementation; verify the other entries against the system descriptions.] 10
3 Functional requirements. 13
4 Non-functional requirements. 14
5 Traceability from requirements to modules and tests. 15
6 Rule families and the principle of their checks. 17
7 Inference rules occurring in `PRV010+1.s` and their intended reading. 18
8 Verdicts, printed SZS status, and score under the ProoVer 2026 scheme. 19
9 Main design decisions and the alternatives considered. 20
10 Module structure of the implementation (approximate line counts). 21
11 Groups of the TST set. 26
12 Verdicts on the ProoVer set (number of proofs) and resulting score. 27
13 Results on the TST set by group (agreeing / total). 27
14 Rules in the ProoVer set and how their steps were decided. 28
15 Size and test coverage of the checker modules. 28
16 Preliminary average run time per check category (five proofs each; to be repeated on the final version and the final hardware). 29
17 Mutation operators and kill rates. 30
18 Causes of suboptimal verdicts on the ProoVer set. 30

References

Proof verification with GDV and LambdaPi – it's a matter of trust. *Proceedings of flairs*. TODO: Autoren, Jahr und Band von journals.flvc.org/FLAIRS/article/view/138642 uebernehmen.

2026. Lean on vampire proofs. TODO: Autoren aus arXiv uebernehmen.

2026. Proceedings of ProoVer 2026. `https://proover-competition.github.io/assets/ProoVer2026_proceedings.pdf`.

ANDREOTTI, BRUNO; HANNA LACHNITT; und HANIEL BARBOSA. 2023. Carcara: An efficient proof checker and elaborator for SMT proofs in the Alethe format. *Tacas 2023*, *LNCS*, Ausg. 13993, 367–386. Springer.

BÖHME, SASCHA, und TOBIAS NIPKOW. 2010. Sledgehammer: Judgement day. *Ijcar 2010*, *LNCS*, Ausg. 6173, 107–121. Springer.

BUTT, MIRIAM, und WILHELM GEUDER (Hg.) 1998. *The projection of arguments: Lexical and compositional factors*. Stanford, CA: CSLI Publications.

CAILLER, JULIE, und SIMON GUILLOUD. 2026. ProoVer: Website of the ProoVer competition. `https://proover-competition.github.io/`. Annual competition for proof checkers in the TPTP ecosystem. Accessed: 2026-10-09.

CROFT, WILLIAM. 1998. Event structure in argument linking. In Butt & Geuder, 21–63.

DELAHAYE, DAVID; DAMIEN DOLIGEZ; FRÉDÉRIC GILBERT; PIERRE HALMAGRAND; und OLIVIER HERMANT. 2013. Zenon modulo: When achilles outruns the tortoise using deduction modulo. *Lpar-19*, *LNCS*, Ausg. 8312. Springer.

DONOHUE, MARK. 2009. Geography is more robust than linguistics. Science e-letter, 13 August 2009. URL `http://www.sciencemag.org/cgi/eletters/324/5926/464-c`.

DORIAN, NANCY C. (Hg.) 1989. *Investigating obsolescence*. Cambridge: Cambridge University Press.

EBNER, GABRIEL; STEFAN HETZL; GISELLE REIS; MARTIN RIENER; SIMON WOLFSTEINER; und SEBASTIAN ZIVOTA. 2016. System description: GAPT 2.0. *Ijcar 2016*, *LNCS*, Ausg. 9706, 293–301. Springer.

FITTING, MELVIN. 1996. *First-order logic*, 109–136. New York, NY: Springer New York. URL `https://doi.org/10.1007/978-1-4612-2360-3_5`.

GROPEN, JESS; STEVEN PINKER; MICHELLE HOLLANDER; RICHARD GOLDBERG; und RONALD WILSON. 1989. The learnability and acquisition of the dative alternation in English. *Language* 65.203–57.

HALE, KENNETH, und JOSIE WHITE EAGLE. 1980. A preliminary metrical account of Winnebago accent. *International Journal of American Linguistics* 46.117–32.

HASPELMATH, MARTIN. 1993. *A grammar of lezgian*. Walter de Gruyter.

HYMES, DELL H. 1974a. *Foundations in sociolinguistics: An ethnographic approach*. Philadelphia: University of Pennsylvania Press.

HYMES, DELL H. (Hg.) 1974b. *Studies in the history of linguistics: Traditions and paradigms*. Bloomington: Indiana University Press.

HYMES, DELL H. 1980. *Language in education: Ethnolinguistic essays*. Washington, DC: Center for Applied Linguistics.

KOVÁCS, LAURA, und ANDREI VORONKOV. 2013. First-order theorem proving and Vampire. *Cav 2013*, *LNCS*, Ausg. 8044, 1–35. Springer.

MINER, KENNETH. 1990. Winnebago accent: The rest of the data. Lawrence: University of Kansas, MS.

MORGAN, TJH; NT UOMINI; LE RENDELL; L CHOUINARD-THULY; SE STREET; HM LEWIS; CP CROSS; C EVANS; R KEARNEY; I DE LA TORRE; ET AL. 2015. Experimental evidence for the co-evolution of hominin tool-making teaching and language. *Nature communications* 6.

PERLMUTTER, DAVID M. 1978. Impersonal passives and the unaccusative hypothesis. *Berkeley Linguistics Society* 4.157–89.

POSER, WILLIAM. 1984. *The phonetics and phonology of tone and intonation in Japanese*. Cambridge, MA: MIT Dissertation.

PRINCE, ELLEN. 1991. Relative clauses, resumptive pronouns, and kind-sentences. Paper presented at the annual meeting of the Linguistic Society of America, Chicago.

RIAZANOV, ALEXANDRE, und ANDREI VORONKOV. 2002. The design and implementation of VAMPIRE. Ausg. 15, 91–110.

RICE, KEREN. 1989. *A grammar of Slave*. Berlin: Mouton de Gruyter.

SALTZMAN, ELLIOT; HOSUNG NAM; JELENA KRIVOKAPIC; und LOUIS GOLDSTEIN. 2008. A task-dynamic toolkit for modeling the effects of prosodic structure on articulation. *Proceedings of the 4th International Conference on Speech Prosody (Speech Prosody 2008)*, Campinas, 175–84. URL `http://aune.lpl.univ-aix.fr/~sprosig/sp2008/papers/3inv.pdf`.

VAN DER SANDT, ROB A. 1992. Presupposition projection as anaphora resolution. *Journal of Semantics* 9.333–77.

SCHULZ, STEPHAN; SIMON CRUANES; und PETAR VUKMIROVIĆ. 2019. Faster, higher, stronger: E 2.3. *Cade-27*, *LNCS*, Ausg. 11716, 495–507. Springer.

SCHURR, HANS-JÖRG; MATHIAS FLEURY; HANIEL BARBOSA; und PASCAL FONTAINE. 2021. Alethe: Towards a generic SMT proof format. *Pxtp 2021*, *EPTCS*, Ausg. 336, 49–54.

SINGLER, JOHN VICTOR. 1992. Review of Melanesian English and the Oceanic substrate, by Roger M. Keesing. *Language* 68.176–82.

SPROUSE, JON; MATT WAGERS; und COLIN PHILLIPS. 2011. A test of the relation between working memory capacity and syntactic island effects. *Language*, to appear.

STOCKWELL, ROBERT P. 1993. Obituary of Dwight L. Bolinger. *Language* 69.99–112.

SUNDELL, TIMOTHY R. 2009. Metalinguistic disagreement. Ann Arbor: University of Michigan, MS. URL `http://faculty.wcas.northwestern.edu/~trs341/papers.html`.

SUTCLIFFE, G. 2006a. Proceedings of the 3rd IJCAR ATP System Competition. Seattle, USA.

SUTCLIFFE, G. 2006b. Report on the 19th International FLAIRS Conference. *AI Magazine* 27.111–112.

SUTCLIFFE, G. 2006c. Semantic Derivation Verification: Techniques and Implementation. *International Journal on Artificial Intelligence Tools* 15.1053–1070.

SUTCLIFFE, G. 2006d. The CADE-20 Automated Theorem Proving Competition. *AI Communications* 19.173–181.

SUTCLIFFE, G. 2007. TPTP, TSTP, CASC, etc. *Proceedings of the 2nd International Computer Science Symposium in Russia*, hrsg. von V. Diekert, M. Volkov, und A. Voronkov, Nr. 4649 in Lecture Notes in Computer Science, 7–23. Springer-Verlag.

SUTCLIFFE, G. 2008. The SZS Ontologies for Automated Reasoning Software. *Proceedings of the LPAR Workshops: Knowledge Exchange: Automated Provers and Proof Assistants, and the 7th International Workshop on the Implementation of Logics*, hrsg. von G. Sutcliffe, P. Rudnicki, R. Schmidt, B. Konev, und S. Schulz, Nr. 418 in CEUR Workshop Proceedings, 38–49.

SUTCLIFFE, G. 2010. The TPTP World - Infrastructure for Automated Reasoning. *Proceedings of the 16th International Conference on Logic for Programming Artificial Intelligence and Reasoning*, hrsg. von E. Clarke und A. Voronkov, Nr. 6355 in Lecture Notes in Artificial Intelligence, 1–12. Springer-Verlag.

SUTCLIFFE, G. 2016. The CADE ATP System Competition - CASC. *AI Magazine* 37.99–101.

SUTCLIFFE, G. 2024. Stepping Stones in the TPTP World. *Proceedings of the 12th International Joint Conference on Automated Reasoning*, hrsg. von C. Benzmüller, M. Heule, und R. Schmidt, Nr. 14739 in Lecture Notes in Artificial Intelligence, 30–50.

SUTCLIFFE, GEOFF. 2000. System description: SystemOnTPTP. *Cade-17*, *LNCS*, Ausg. 1831, 406–410. Springer.

SUTCLIFFE, GEOFF. 2006e. Semantic derivation verification: Techniques and implementation. *International Journal on Artificial Intelligence Tools* 15.1053–1070.

SUTCLIFFE, GEOFF. 2017. The TPTP problem library and associated infrastructure: From CNF to TH0, TPTP v6.4.0. *Journal of Automated Reasoning* 59.483–502.

SUTCLIFFE, GEOFF; STEPHAN SCHULZ; KOEN CLAESSEN; und ALLEN VAN GELDER. 2006. Using the TPTP language for writing derivations and finite interpretations. *Ijcar 2006*, *LNCS*, Ausg. 4130, 67–81. Springer.

TAPROGGE, ET AL. 2026. Nörgler: A light-weight proof checker for TSTP refutations. *Practical aspects of automated reasoning (paar)*, *CEUR Workshop Proceedings*, Ausg. 4241. TODO: vollstaendige Autorenliste (Taprogge, Sariyanto, Steen) und Seiten pruefen.

TIERSMA, PETER M. 1993. Linguistic issues in the law. *Language* 69.113–37.

WETZLER, NATHAN; MARIJN J. H. HEULE; und WARREN A. HUNT JR. 2014. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. *Sat 2014*, *LNCS*, Ausg. 8561, 422–429. Springer.

WILSON, DEIRDRE. 1975. *Presuppositions and non-truth-conditional semantics*. London: Academic Press.

YIP, MOIRA. 1991. Coronals, consonant clusters, and the coda condition. *The special status of coronals: Internal and external evidence*, hrsg. von Carole Paradis und Jean-François Prunet, 61–78. San Diego, CA: Academic Press.